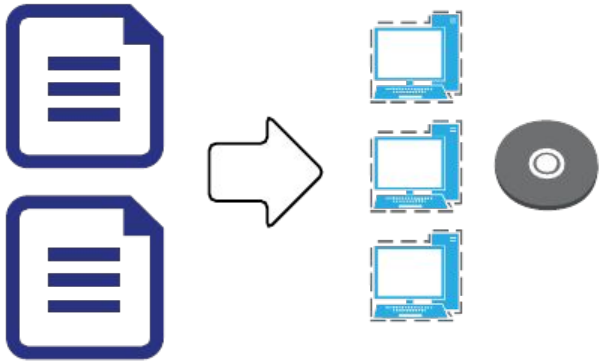
Store Distribuido

Almacenamiento Distribuido

Lo que se quiere resolver al utilizar almacenamiento distribuido es poder almacenar información que por su volumen no entra en un nodo servidor.

Este volumen puede estar dado porque:

- Un elemento por sí solo no entra en un nodo.
- El volumen de elementos es tal, que la suma de sus tamaños supera los nodos.



Almacenamiento Distribuido

Se hace mención de "elementos" porque existen herramientas y frameworks que permiten distribuir diferentes tipos de valores, que dan la oportunidad de utilizar abstracciones mayores a la hora de guardar.

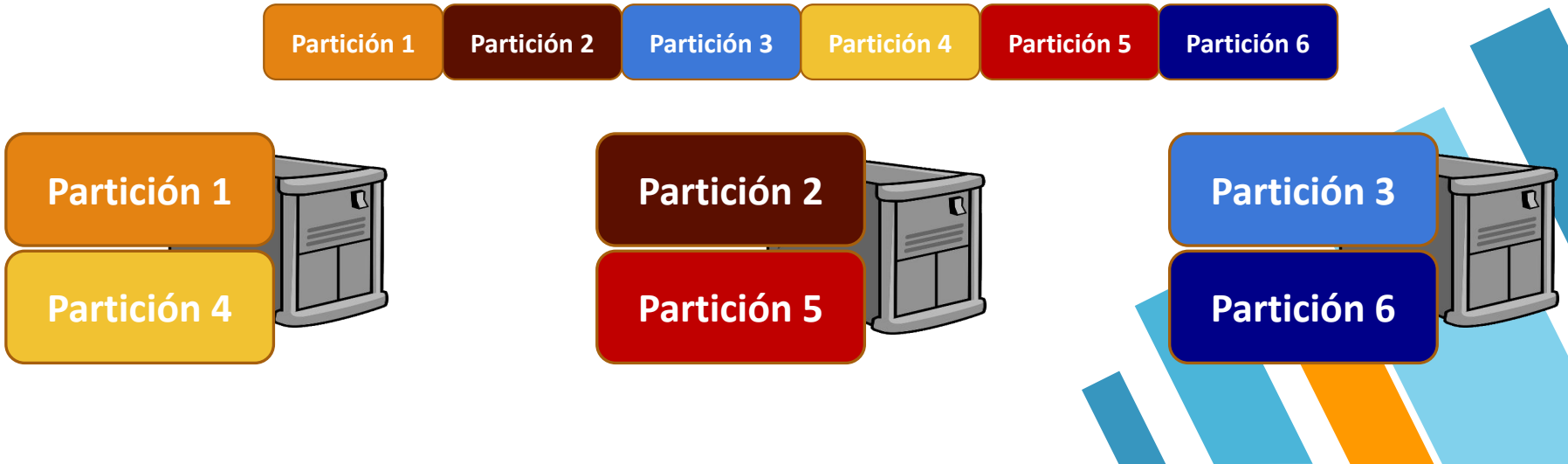
Por lo cual se encuentran *stores* distribuidos de diferentes tipos:

- File Systems
- Bases de Datos
- Colecciones de Objetos.

Particionado

El primer inconveniente a resolver es cómo distribuir la información dentro del cluster.

Teniendo varios nodos dentro del cluster lo más lógico es "dividir" la tarea (almacenar) entre los nodos. O sea particionar la información.



Particionado - Distribución

Una buena partición de los datos nos permite

- Realizar una escritura más rápida y eficiente.
- Que el lookup del dato sea más rápido.

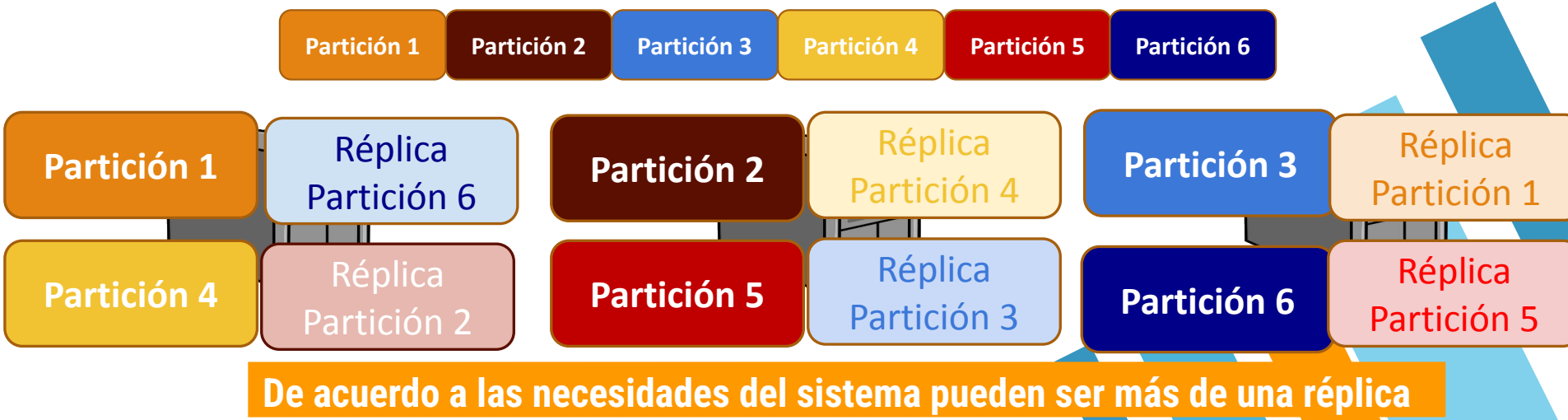
Tener en cuenta tiene sus contras:

- Hay que coordinar el cluster para saber dónde va cada partición y a cuál partición va cada valor.
- De elegir mal la estrategia de partición, los datos pueden quedar sesgados en uno o más servidores, haciendo que la carga de los mismos lo haga ineficiente.

Replicación

Tener una copia de cada partición en el sistema es peligroso, porque si **se cae un nodo, la información queda corrupta** (esa partición dejó de existir)

Para resolver este potencial problema **se tienen varias copias** de cada partición dentro del cluster o sea se **replican las particiones**.



De acuerdo a las necesidades del sistema pueden ser más de una réplica

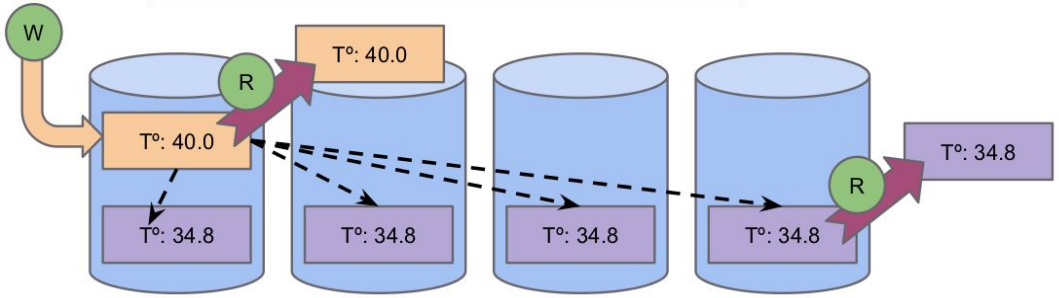
Consistencia

Al tener particionado y replicación, una decisión que hay que tomar es cuando considero al dato escrito (y cuándo se lo puede leer).

Algunas opciones pueden ser:

1. Al momento que se escribió en la partición principal inicial.
2. Cuando hay al menos una réplica escrita + la partición.
3. Cuando estén todas las réplicas escritas.

E inclusive una decisión más a realizar ¿se puede leer o escribir en las réplicas?



Consistencia

Cualquiera de las soluciones es válida, de acuerdo a las necesidades del sistema a implementar.

- En el caso de necesitar mucha capacidad de lectura y data consistente (similar a una base de datos ACID), la opción 3 será la elegida.
- En cualquier otro caso, 1 y 2 son opciones válidas.

Consistencia

De la misma manera que se puede configurar la cantidad de réplicas y particiones existen dos factores que indican la preferencia en la consistencia:

- **Consistencia en escritura:** Indica cuántas copias tienen que reportar que la escritura fue hecha antes de que se le indique al cliente que la escritura está terminada.
- **Consistencia en lectura:** Indica cuántas particiones tienen que reportar el mismo valor antes de responder ese valor como cierto en una lectura.

Modelos de Consistencia

Los modelos de consistencia se utilizan en sistemas de store distribuidos y de memoria distribuida compartida (DSM).
Cada modelo indica una serie de reglas para la lectura/escritura de un valor, que asegura que, si el programador sigue las reglas, sus lecturas serán consistentes con las escrituras hechas.

Modelos de Consistencia

Modelo de consistencia estricto
Es el modelo ideal, por el cual toda escritura estará inmediatamente disponible para lectura para todos. Además se respetará el orden de cada operación de manera global.

Modelo de consistencia eventual
Indica que luego de realizar una escritura, y de no mediar nuevas escrituras, pasado cierto tiempo todas las lecturas reflejarán el valor escrito.

Modelos de Consistencia

Los dos modelos mencionados son los más comunes en store distribuido.
Los modelos de consistencia son muchos y tienen diversos campos de aplicación. Una lista no completa de los mismos se puede hallar en la wiki del tema.

Elasticidad

La elasticidad es la capacidad de un cluster de modificar la cantidad de nodos que lo componen y que éstos puedan participar de la operación del mismo **sin que el cluster quede offline** completamente.

En un cluster de storage se requería que:

- El cluster detecte al nuevo nodo.
- Le asigne particiones primarias y de backup.
- Coordine la transferencia de esas particiones.
- Lo marque disponible para continuar como parte del proceso.

Tolerancia a Fallos

Parte de las tareas de controlar es que, **si algún nodo se cae, las particiones** del mismo (primarias y de backup) **sean reasignadas.**

Todo esto **requiere coordinación** y hay varias estrategias para que esto sea posible, en general hay **un nodo master o coordinador** y alguna redundancia (o no).

Lo que varía es si el master es un nodo especial o cualquier nodo puede asumir el rol de coordinador

HDFS

Hadoop Distributed File System

Hadoop: componentes

Hadoop Distributed File System (HDFS™):
File System distribuido



Hadoop MapReduce

Framework de procesamiento distribuido



Hadoop YARN

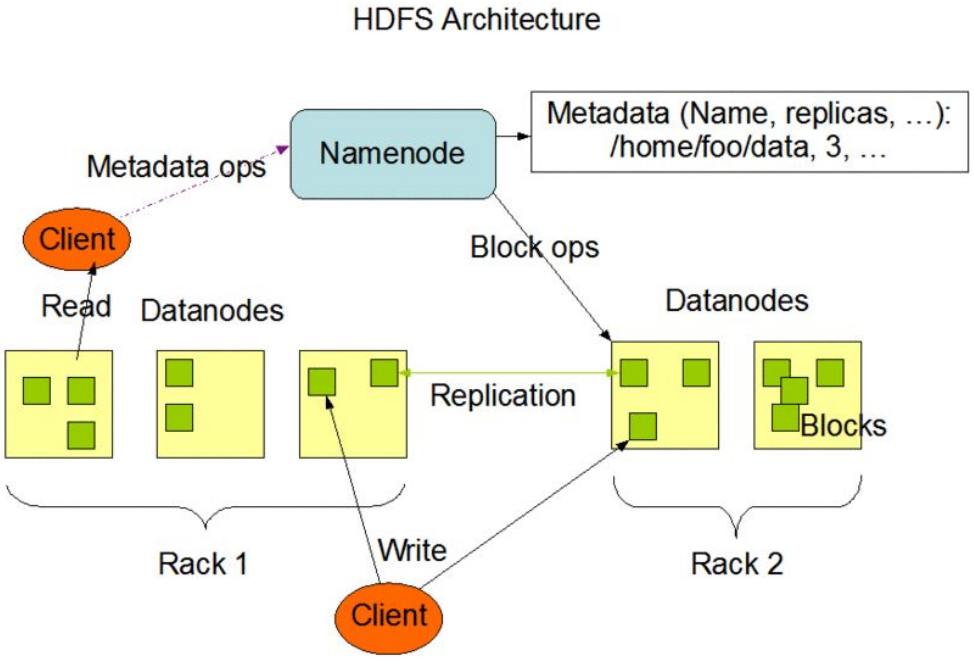
Framework de manejo de recursos y tareas distribuidas



HDFS: Características.

- **File system distribuido** modelado a partir del paper Google File System.
- Optimizado para **high throughput** y trabaja mejor **leyendo y escribiendo archivos muy grandes** (órdenes de Gigas o más)
- Utiliza tamaños de bloque muy grande.
- Maneja replicación para lograr **fault tolerance**.
- Diseñado para correr en **commodity hardware** (no RAID, NFS, etc.)

Hadoop: arquitectura

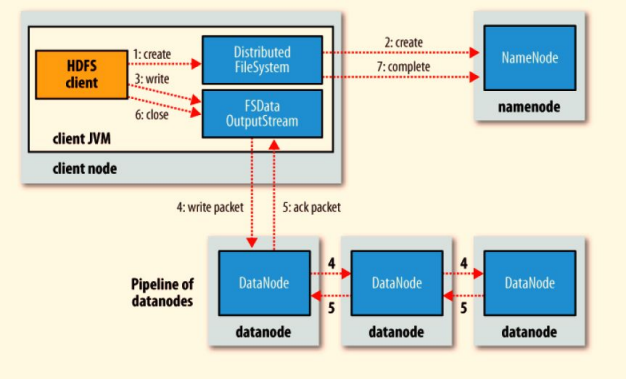


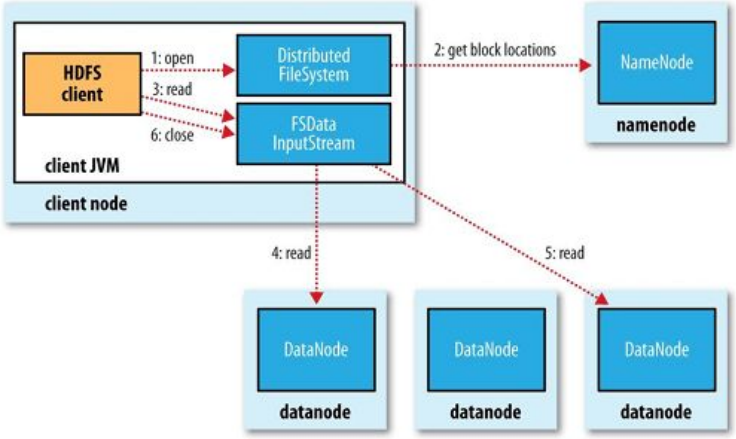
HDFS: Bloque

- Los archivos en HDFS se particiones **en bloques de tamaño definido en configuración** (por defecto de 128MB)
- Cada bloque es **replicado** en diferentes nodos para redundancia. La cantidad de réplicas es configurable
- Si el archivo es menor que un bloque, solo **ocupa su tamaño real**.
- Al particionar en bloques **se simplifica el manejo de archivos** porque al tener tamaño máximo un nodo puede manejar un bloque.

HDFS: Name Node

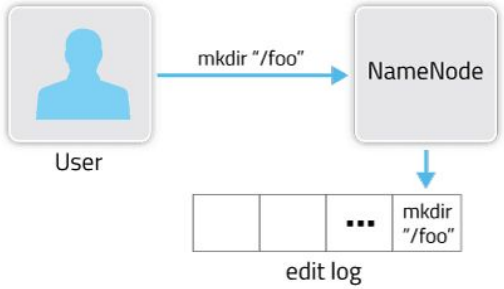
- **Administra el namespace del file system** y regula el acceso de los clientes.
- **Administra y mantiene la estructura lógica y metadata de carpetas y archivos** (permisos tamaños y ubicaciones)**.** Usa la estructura Linux.
- Relaciona archivos con sus bloques
- **Conoce el estado de los Data Nodes**
- Responde los pedidos de búsqueda y escrituras de bloques .

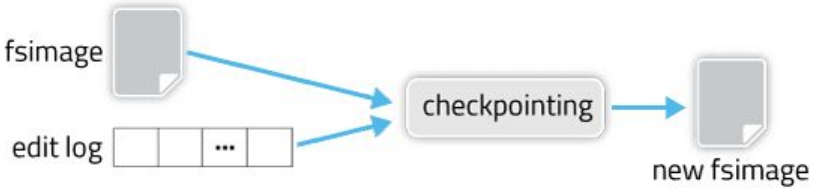




HDFS: Name Node

- **Persiste la información en el disco**. A partir de dos componentes
  - FsImage: Snapshot que se toma cada tanto del estado del file system
  - Transaction log: estructura que guarda las escrituras hasta que la FsImage se actualice.





HDFS: Data Nodes.

- **Guarda los bloques de información** que son parte de los archivos guardados en el sistema.
- **Se reporta al Name Node** periódicamente (default 3 segundos) para informar que sigue vivo y cuales son los bloques que tiene.
- **Se comunica con los otros Data nodes** para transmitir la información que se está replicando un nodo.

HDFS: Problemas

- **El Name Node es un SPOF** (single point of failure)
  - Primera aproximación de solución en HDFS 2.X con “federación”
  - Solución definitiva cuando se agregó HA en 3.X.
- **Problemas con el acceso random a la información de los archivos**
  - Cómo está optimizado para leer por bloque, acceder a un byte en el medio no es la mejor manera de usarlo.
  - Se crearon bases de datos como HBASE, para agregarle una capa de acceso al HDFs.
- **Muchos archivos pequeños degradan la performance.**
  - Se deben intentar evitar, si es posible mergear o particionar de manera que no afecte a la performance.

¿Cómo se usa?

```
$> hadoop fs -command <args>
```

```
$> hadoop fs -get hdfs://nn.example.com/user/hadoop/file localfile #Copy files to the local system.
$> hadoop fs -put -f localfile1 localfile2 /user/hadoop/hadoopdir #Copy from local to destination system
$> hadoop fs -cp /user/hadoop/file1 /user/hadoop/file2 #copy from source to destination.
$> hadoop fs -mv hdfs://nn.example.com/file1 hdfs://nn.example.com/file2 #Moves from source to destination.
$> hadoop fs -mkdir /user/hadoop/dir1 #creates directories.
$> hadoop fs -ls /user/hadoop/file1 #returns list of its direct children
$> hadoop fs -rm hdfs://nn.example.com/file /user/hadoop/emptydir #Delete files specified.
$> hadoop fs -rmdir /user/hadoop/emptydir #Delete a directory.
$> hadoop fs -tail pathname #Displays last kilobyte of the file to stdout.
$> hadoop fs -getmerge -nl /src /opt/output.txt #Takes a source directory and a destination file as input and concatenates files in src into the destination local file.
```

Más comandos

Hazelcast

Hazelcast

Es un *grid* de datos en memoria, distribuido y *open source*.
Realizado en Java, con mínimas dependencias y la posibilidad de correr embebido o mediante una arquitectura *client-server*

Hazelcast- Modelo de datos

Hazelcast **redefine varias estructura de datos,** respetando la interfaz conocida de Java para distribuir colecciones:

- IMap
- IList
- IQueue
- ITopic

Estas estructuras de datos son las que tenemos que usar si queremos que la *data* sea distribuida.

Si usamos las estructuras tradicionales sólo las usa el nodo que las declara.

Hazelcast - Casos de Uso

Principales casos de uso en los cuales Hazelcast puede ser útil:

- In-Memory Data Grid
- Caching
- Microservices
- Web Session Clustering
- Messaging
- In-Memory NoSQL
- Application Scaling
- Procesamiento de operaciones distribuidos

Ejercicio 1 - Instanciar un *cluster*

Hazelcast trabaja con un jar simple sin otras dependencias para instanciar un nodo.

- Descargar de Campus el archivo **`hazelcast-all-3.8.6.jar`**
- Definir el *classpath* para que apunte al jar.
- Ejecutar la aplicación en 2 consolas.

```
$>  java -jar hazelcast-all-3.8.6.jar
```

O

```
$> export CLASSPATH=$HOME/hazel/hazelcast-all-3.8.6.jar
$> java com.hazelcast.console.ConsoleApp
```

Hazelcast - Uso

El nodo se inicializa y, a medida que se van incorporando los miembros, aparecen en el listado y se puede utilizar la consola para realizar operaciones.

```
may 05, 2026 8:27:24 P. M. com.hazelcast.system
INFO: [192.168.0.90]:5702 [dev] [3.8.6] Cluster version set to 3.8
may 05, 2026 8:27:24 P. M. com.hazelcast.internal.cluster.ClusterService
INFO: [192.168.0.90]:5702 [dev] [3.8.6]
Members [2] {
     Member [192.168.0.90]:5701 - 39609930-8e74-4236-9053-0a3691dc8ae1
     Member [192.168.0.90]:5702 - 617115c1-3934-4c56-9c1e-77c9cb706b2c this
}
may 05, 2026 8:27:26 P. M. com.hazelcast.core.LifecycleService
INFO: [192.168.0.90]:5702 [dev] [3.8.6] [192.168.0.90]:5702 is STARTED
hazelcast[default] >
```

Ejercicio 2 - Uso de la consola

- Desde la consola el comando **`help`** permite ver cuáles son los comandos disponibles
- El **[default]** indica el ***namespace*** (nombre de la colección que estamos usando).
- Insertar en el mapa para la clave ***alumno*** su legajo como valor y luego verificar el valor asociado a dicha clave.

```
hazelcast[default] > m.put  alumno 12345
null
hazelcast[default] > m.get  alumno
12345
```

- Cambiar el *namespace* al legajo en cuestión para no pisarse y repetir

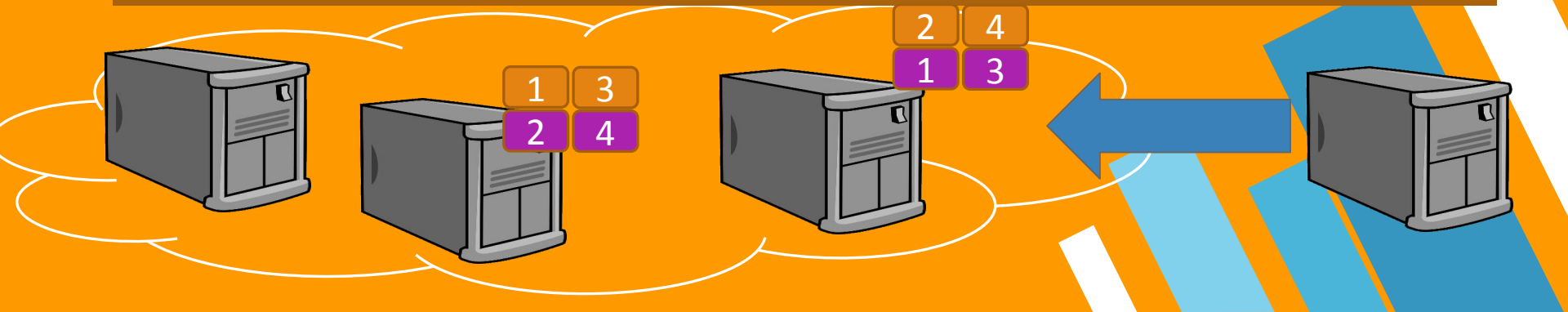
```
hazelcast[default] > ns 12345
namespace: 12345
hazelcast[12345] > ...
```

Hazelcast - Tipos de nodo

***Node Member*:** Es el tipo de nodo por *default*. Estos tienen datos distribuidos y además realizan tareas distribuidas.

***Lite Member*:** Son nodos que no guardan información, sirven para lanzar tareas y otro tipo de acciones (como agregar *listeners*).

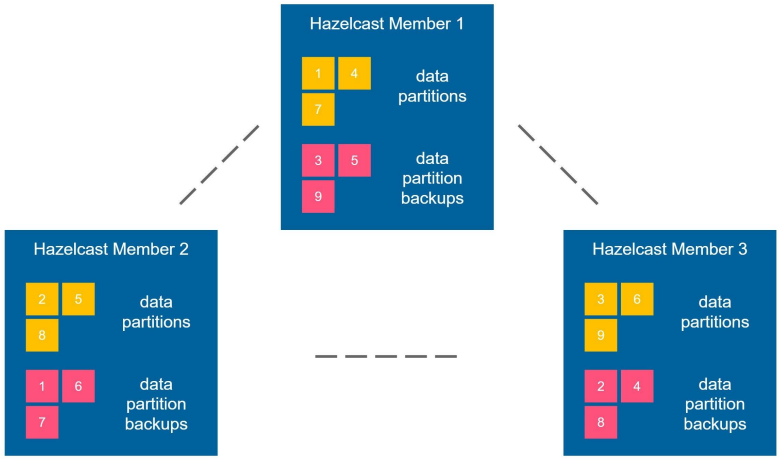
***Native Client*:** No es un nodo del *cluster*, sino una manera en la que me puedo conectar a un *cluster* de manera Cliente/Servidor. Este cliente se utiliza para poner datos distribuidos dentro del *cluster*, pedir ejecutar cosas en el *cluster*.



Hazelcast - Coordinación

El *cluster* de Hazelcast tiene una arquitectura ***peer-to-peer***, no hay un *master*, sino que cualquier nodo puede asumir el control. Entre otros beneficios elimina la posibilidad de un *Single Point Of Failure*.

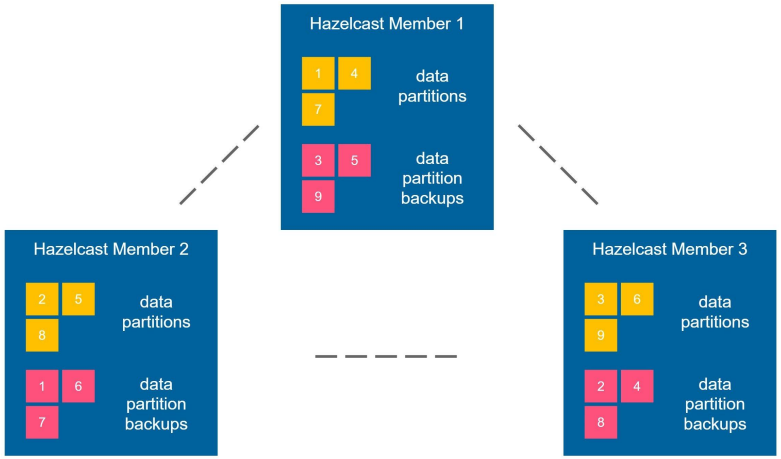
Para la coordinación y asignación de trabajo y particiones **se selecciona al nodo de mayor antigüedad del *cluster* como Coordinador**.



Hazelcast - Coordinador

El coordinador **posee una tabla de particiones donde dice qué nodo es *owner/backup*** de qué particiones, pero la distribuye entre todos. O sea, al llegar/desaparecer un nodo la actualiza y la vuelve a distribuir.

En caso de que el coordinador se caiga se selecciona al siguiente nodo más viejo como nuevo coordinador y se recalcula la tabla de particiones.

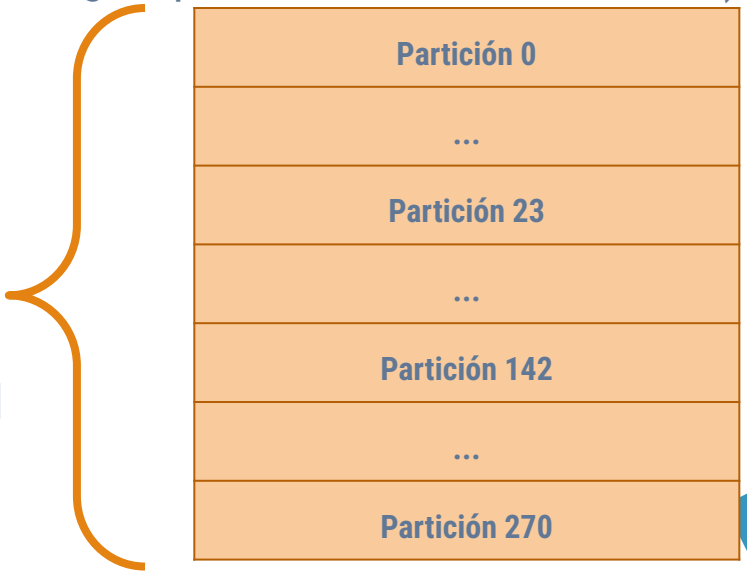


Hazelcast - Particionado

La estructura global se divide **en 271 particiones** entre los nodos del *cluster*, con posibilidad de redundancia (réplicas o *backups*) entre nodos.

Aunque los datos están distribuidos entre nodos, cualquier nodo accede a la totalidad de los datos (si no los tiene él igual puede solicitar a otros nodos).

271 particiones primarias para distribuir entre los nodos del *cluster*

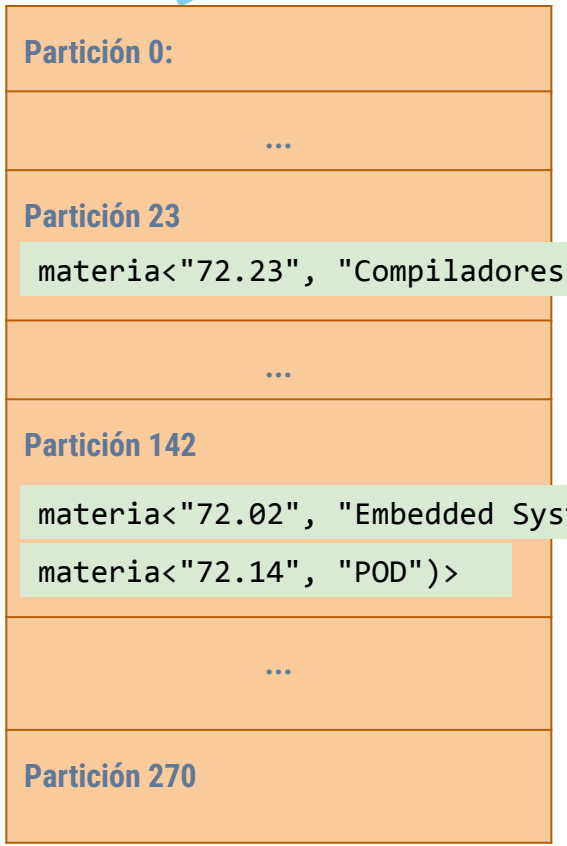


Hazelcast - Asignación a Particiones

Para decidir qué partición es owner de qué objeto, usa el algoritmo “Consistent Hashing”.

Nro. Partición(objeto) = hash(objeto) % 271

Hazelcast - Asignación a Particiones



Suponiendo las siguientes inserciones:

```
Map<String, String> materias =
        hz.getMap("materias");
materias.put("72.02", "Embedded Systems");
materias.put("72.23", "Compiladores");
materias.put("72.14", "POD");
```

hashcode("72.02")%271 = 142
hashcode("72.23")%271 = 23
hashcode("72.14")%271 = 142

Hazelcast - Réplicas

¿Si hay un solo nodo, hay *backup*?

Rta: No

¿Si hubiera 2 nodos y *backup*, cuántas particiones les toca a cada uno de esos 2 nodos?

Rta: aprox. 271/2 primarias y 271/2 *backup* (del otro nodo)

¿Si hubieran 3 nodos, cuántas particiones les tocaría a cada uno de esos 3 nodos?

Rta: aprox. 271/3 primarias y 271/3 *backup* (de los otros nodos)

Hazelcast - Réplicas

- **Las réplicas se actualizan sincrónicamente** con la partición principal.
- Por *default*, **hay una única réplica** porque ese sincronismo provoca una caída en la performance pero **la cantidad de réplicas puede aumentarse o reducirse**.

Hazelcast - Réplicas Recupero

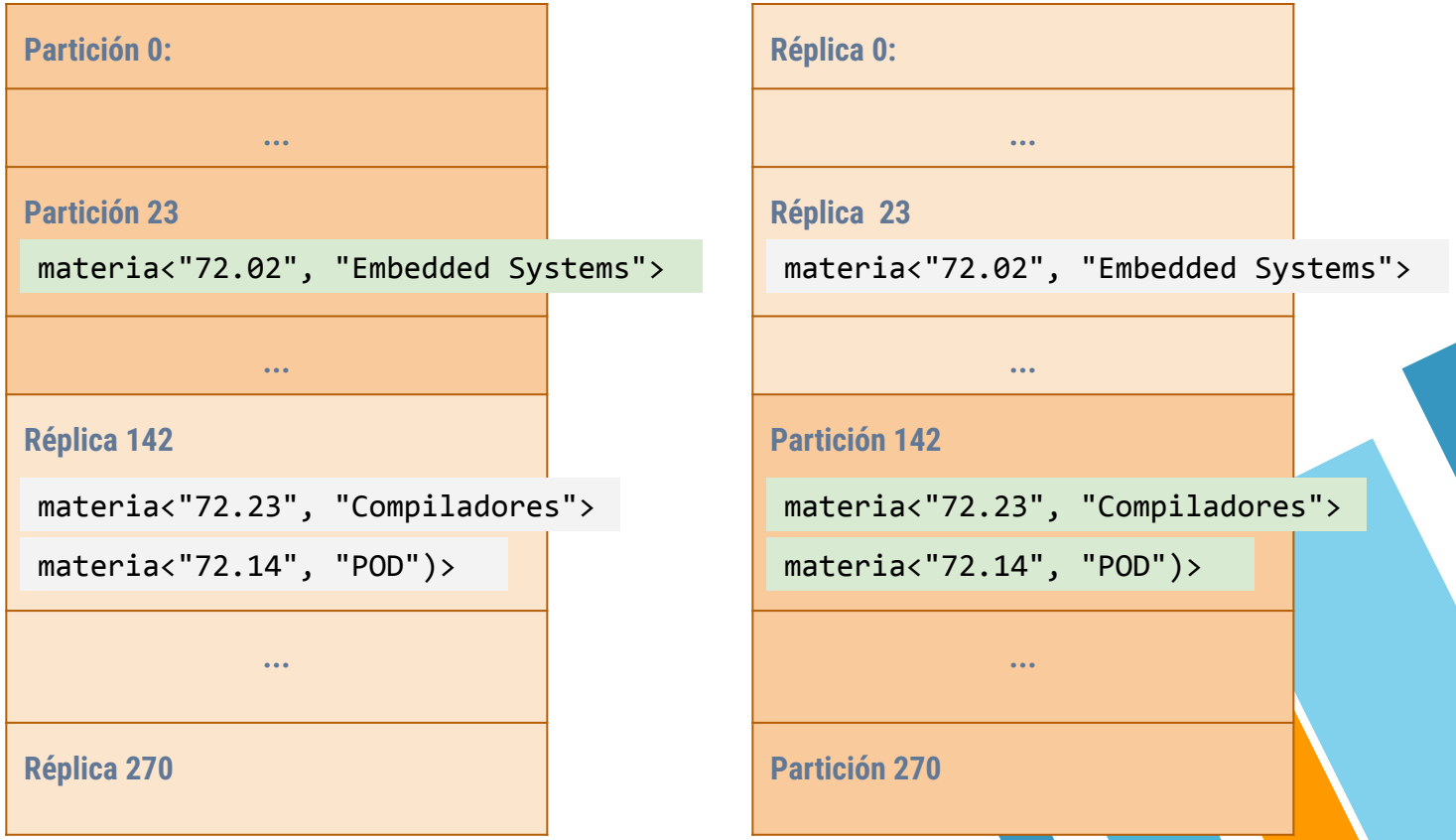
- Ante **la caída del nodo que es *owner*** de cierta partición (tiene la copia principal), **el nodo que contiene su réplica pasa a ser el nuevo *owner*** y su réplica pasa a ser la principal. En dicho caso otro nodo será asignado para mantener una réplica de la partición.
- Al agregarse o retirarse nodos se inicia la migración de las particiones, de manera que nunca principal y réplica queden en el mismo nodo.

Hazelcast - Migración de Particiones

Dado el caso anterior, si agregamos un nodo nuevo:



Hazelcast - Migración de Particiones



Hazelcast - Réplicas Asincrónicas

En caso de querer que la escritura responda con mayor velocidad se puede configurar para que **el *backup* se haga de manera asincrónica**.

Para estas réplicas asincrónicas se garantiza “consistencia eventual”. Obviamente es más performante, pero puede haber problemas con los datos obtenidos, ya que se puede leer un dato *stale* (desactualizado).

Hazelcast - Configuración

Hazelcast- Configuración

Para poder configurar el *cluster* y que no se tomen valores por defecto hay varias maneras de configurarlos:

- Declarativamente a través de un archivo XML (`hazelcast.xml`)
- Programáticamente desde el cliente Java
- Utilizando propiedades del sistema
- Usando contexto de Spring

Vamos a ver el modo declarativo en XML.

Hazelcast - Member Discovery

Para que los miembros del *cluster* se encuentren hay varios modos configurables:

- **Multicast**
- **TCP/IP**
- ***Discovery* con *frameworks* como Zookeeper, Consul**
- ***Discovery* en *clouds* como AWS, AZURE**

```
//MULTICAST
<multicast enabled="true">
   <multicast-group>224.2.2.3</multicast-group>
   <multicast-port>54327</multicast-port>
   <multicast-time-to-live>32</multicast-time-to-live>
   <multicast-timeout-seconds>2</multicast-timeout-seconds>
   <trusted-interfaces>
       <interface>192.168.1.102</interface>
   </trusted-interfaces>
</multicast>
       enabled="false"/>
<aws enabled="false"/>
```

```
//TCP IP
<multicast enabled="false" />
<tcp-ip enabled="true">
<member>machine1</member>
<member>machine2</member>
<member>machine3:5799</member>
<member>192.168.1.0-7</member>
<member>192.168.1.21</member>
</tcp-ip>
```

El modo *multicast* no es recomendado en entornos productivos ya que la comunicación vía UDP puede ser problemática y generalmente se bloquea.

Hazelcast - Cluster Security

Aún con la misma configuración de *discovery* se pueden tener *clusters* independientes entre sí. Para ello, dentro de la configuración, se puede setear un "grupo" específico para cada *cluster*.

En la misma se indica un nombre y una *password* para poder unirse al *cluster*, por lo que también provee una cierta "seguridad" al hecho de poder unirse al grupo.

```
<group>
    <name>dev</name>
    <password>dev-pass</password>
</group>
```

Hazelcast - Configuración de colecciones

Además de la configuración de red y *clusters* se puede configurar cada colección y otros elementos como el Executor Service.

```
<map name="default">
   <backup-count>1</backup-count>
   <async-backup-count>1</async-backup-count>
   <read-backup-data>false</read-backup-data>
</map>

<map name="materias">
<backup-count>2</backup-count>
<async-backup-count>1</async-backup-count>
<read-backup-data>true</read-backup-data>
</map>
```

En el ejemplo se configuran 2 mapas "**default**" y "**materias**".

- Para **default** se crean 1 partición de *backup* sincrónica, 1 asincrónica y no se permite la lectura de las particiones de *backup*.
- Para **materias** se configuran 2 *backups* sincrónicos y 1 asincrónico y se puede leer de los mismos.

Hazelcast - Split Brain

Dado un *cluster* se da un ***split brain*** cuando por problemas de comunicación el mismo se subdivide entre dos partes autónomas.

Esto puede causar graves fallas, porque:

- Ambos *clusters* siguen funcionando
- Pueden recibir actualizaciones que conflictúan (por ejemplo ambos reciben dos valores para la misma *key*)
- Si los "clusters" se vuelven a reconectar, cómo se realiza el *merge* de los datos

Hazelcast - Split Brain

Para resolver un *split brain* en Hazelcast se realizan los siguientes pasos:

1. Elige un líder entre los dos líderes de los *cluster* (elige al más viejo).
2. Se decide cuál de los dos *clusters* es el que debe *mergearse* (el más chico o por una función).
3. Por cada miembro del *cluster* a mergear:
    a. Pausar su operación (no acepta escrituras ni lecturas).
    b. Cierra la conexión al *cluster* que pertenece.
    c. Se une a otro *cluster*.
    d. Envía un request para *mergear* los valores de los mapas de los cuales es dueño, según la política indicada. Para otras colecciones (listas, colas, etc) no *mergea*, elige una de las dos y se queda con esa.
    e. Reinicia su operación.

Hazelcast - Split Brain

```
<hazelcast>
   ...
   <map name="default">
       <!--
         While recovering from split-brain (network partitioning),
         map entries in the small cluster will merge into the bigger cluster
         based on the policy set here. When an entry merge into the
         cluster, there might an existing entry with the same key already.
         Values of these entries might be different for that same key.
         Which value should be set for the key? Conflict is resolved by
         the policy set here. Default policy is hz.ADD_NEW_ENTRY
         There are built-in merge policies such as
         com.hazelcast.map.merge.PassThroughMergePolicy; entry will be added if
             there is no existing entry for the key.
         com.hazelcast.map.merge.PutIfAbsentMapMergePolicy ; entry will be
             added if the merging entry doesn't exist in the cluster.
         com.hazelcast.map.merge.HigherHitsMapMergePolicy ; entry with the
             higher hits wins.
         com.hazelcast.map.merge.LatestUpdateMapMergePolicy ; entry with the
             latest update wins.
       -->
       <merge-policy>MY_MERGE_POLICY_CLASS</merge-policy>
   </map>
   ...
</hazelcast>
```

Ejercicio 3 - Configuración

- Descargar de Campus el archivo `hazelcast.xml` al CWD
- Modificar el mismo para que
  - el *discovery* se haga por TCP y se encuentren sólo miembros en la computadora local
  - generar un grupo a partir del legajo de cada uno.
- Correr la aplicación miembro en varias consolas
- Comprobar que se conformó el cluster y se puede utilizar sin interferencia de otros

```
$> export CLASSPATH=$HOME/hazel/hazelcast-all-3.8.6.jar
$> java -Dhazelcast.config=$HOME/hazel/hazelcast.xml com.hazelcast.console.ConsoleApp
```

Hazelcast - Java

Hazelcast - Código Java

Para poder correr un nodo en Java se requiere:

- Incorporar la dependencia (alcanza con el -cli, pero si tenemos el -all lo incluye)
- Disponibilizar el xml de configuración (o configurar programáticamente).
- Instanciar al objeto de Hazelcast mediante:

```
//node  client
HazelcastInstance hz = HazelcastClient.newHazelcastClient();

 o

//node member
HazelcastInstance hz = Hazelcast.newHazelcastInstance();
```

A partir de dicha instancia obtener la colección o elemento a utilizar y usarlo.

Hazelcast - Código Java

```
static void main() {
   HazelcastInstance hz = Hazelcast.newHazelcastInstance();

   Map<String, String> datos = hz.getMap("materias");

   datos.put("72.42", "POD");

   IO.println("%d Datos en el cluster".formatted(datos.size()));

   for (String key : datos.keySet())  {
      IO.println("Datos con key %s= %s".formatted(key,
datos.get(key)));
   }
}
```

```
$ java   com.hazelcast.console.ConsoleApp
```

```
hazelcast[default] > ns materias
```

```
hazelcast[materias] > m.put 72.42 POD
```

```
hazelcast[materias] > m.size
```

```
hazelcast[materias] > m.iterator
```

Ejercicio 4 - Código

- **Clonar el repo usando el link de GitHub (Campus)**
- En el módulo `server` los cambios necesarios en el `hazelcast.xml`
- Comprobar que se conformó el cluster y se puede utilizar sin interferencia de otros

Hazelcast - Configuración Java

Hazelcast- Configuración via Java

Tanto *server* como *client* a la hora de instanciarse van a recibir un objeto de configuración al cual se le puede ir agregando mediante *setters* diferentes tipos de configuraciones.

Estas configuraciones pueden ser:

- Seguridad
- Conexión
- Autenticación
- Discovery
- Red
- Colecciones
- etc.

Hazelcast - Server

Para instanciar un *server* necesitamos una instancia de `Config` y simplemente inicializar el *server*.

```
public class Server {
    static void main() {
        // Config
        Config config = new Config();
        // Start cluster
        HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
        // more code.....
        hz.shutdown();
    }
}
```

Hazelcast - Client

Para instanciar un *client* necesitamos un `ClientConfig` para instanciar la `HazelcastClient` que es el punto de entrada.

```
public class Client {

    static void main() {
        // Client Config
        ClientConfig clientConfig = new ClientConfig();
        HazelcastInstance hazelcastInstance =
                                HazelcastClient.newHazelcastClient(clientConfig);
        // more code.....
        HazelcastClient.shutdownAll();
    }

}
```

Hazelcast - Group Config

Tanto cliente como servidor pueden definir una configuración de grupo. Esta configuración sirve como una autenticación para que solo los nodos que pertenecen al mismo grupo formen un cluster. Y los que no pertenecen no pueden formar la configuración.

```
GroupConfig groupConfig = new GroupConfig()
        .setName("cluster").setPassword("cluster-pass");

config.setGroupConfig(groupConfig);
```

Hazelcast - Network Config

Network configuration es la configuración que permite para los nodos del *cluster* saber cómo se conforma el cluster.

```
NetworkConfig networkConfig = new NetworkConfig();

// more code

config.setNetworkConfig(networkConfig);
```

Hazelcast - (Network) Join Config

Join configuration es parte de la configuración de red e indica cómo los nodos del *cluster* van a poder encontrarse entre ellos.

```
MulticastConfig multicastConfig = new MulticastConfig();
JoinConfig joinConfig = new JoinConfig().setMulticastConfig(multicastConfig);
networkConfig.setJoin(joinConfig);
```

Alternativa listando los nodos del *cluster* (más seguro)

```
JoinConfig join = networkConfig.getJoin();
join.getTcpIpConfig().addMember("10.45.67.32").addMember("10.45.67.100")
       .setRequiredMember("192.168.10.100").setEnabled(true);
```

Hazelcast - (Network) Interfaces Config

Permite configurar la interfaz de red que utilizar los nodos del *cluster*

```
InterfacesConfig interfacesConfig = new InterfacesConfig()
        .setInterfaces(List.of("192.168.1.*"))
        .setEnabled(true);
networkConfig.setInterfaces(interfacesConfig);
```

Hazelcast - Network Otras configs

Otras configuraciones de red posibles pueden ser (no todas configurables por código)

- Public Address
- Port
- Outbound Ports
- Reuse Address
- Member Address Provider SPI
- Advanced Network Configuration

Hazelcast - Client Config

La *client config* es la que le permite a un nodo *client* encontrar al *cluster* para realizar pedidos. Internamente maneja elementos homólogos (y algunos iguales) que el servidor

```
ClientConfig clientConfig = new ClientConfig();
// Group Config
GroupConfig groupConfig = new GroupConfig()
        .setName("cluster").setPassword("cluster-pass");
clientConfig.setGroupConfig(groupConfig);
// Client Network Config
ClientNetworkConfig clientNetworkConfig = new ClientNetworkConfig();
clientNetworkConfig.addAddress("192.168.1.51:5701");
clientConfig.setNetworkConfig(clientNetworkConfig);
```

Hazelcast - Management Console

Hazelcast ofrece una *webapp* para monitorear en tiempo real el *cluster*. Nota: Es necesario utilizar una versión coincidente con la de `hazelcast-all` (Link).

```
// Management Center Config
ManagementCenterConfig managementCenterConfig = new ManagementCenterConfig()
        .setUrl("http://localhost:32768/mancenter/")
        .setEnabled(true);
config.setManagementCenterConfig(managementCenterConfig);
```

Hazelcast - Serialización Java

Hazelcast - Serialización

Todas las clases que se quieran introducir en las colecciones deben distribuirse por los nodos vía red.

¿Entonces qué debe poder hacerse con las mismas?

SERIALIZARSE

Hazelcast - Serialización

Para serializar, Hazelcast provee varios caminos:

- Utilizar la interfaz Serializable de Java
- Utilizar la interfaz DataSerializable o IdentifiedDataSerializable de HZ
- Utilizar interfaces alternativas como Portable de HZ o Externalizable de Java
- Implementar serializadores *custom*.

La recomendada es utilizar **DataSerializable** o **IdentifiedDataSerializable** ya que utiliza features de **Java NIO** para hacer más eficiente la serialización.

Hazelcast - Serialización

```java
public class Employee implements DataSerializable {
    private String firstName, lastName;
    private int age;
    private Address address;
    public Employee() {}
    //getters setters..
    @Override
    public void writeData(ObjectDataOutput out) throws IOException {
        out.writeUTF(firstName);
        out.writeUTF(lastName);
        out.writeInt(age);
        address.writeData (out);
    }
    @Override
    public void readData(ObjectDataInput in) throws IOException {
        firstName = in.readUTF();
        lastName = in.readUTF();
        age = in.readInt();
        address = new Address();
        // since Address is DataSerializable let
        // it read its own internal state
        address.readData(in);
    }
}
```

Ejemplo de DataSerializable

Hazelcast - Tipos de Elementos

Hazelcast - Tipos

Hazelcast define tipos que representan colecciones o elementos distribuidos:

- Map
- MultiMap
- Cache
- Queue
- Ringbuffer
- Set
- List
- Replicated Map
- Cardinality Estimator
- Topic
- Lock
- Semaphore
- AtomicLong
- AtomicReference
- IdGenerator
- Countdown Latch

Hazelcast - Tipos Compatibilidad

Cada uno de estos tipos si tiene un equivalente en el lenguaje local intentan mantener la semántica.

En Java por ejemplo el mapa distribuido extiende del mapa de Java por lo que se lo puede usar como si “no fuera distribuido”

```
package com.hazelcast.core;

public interface IMap<K, V> extends ConcurrentMap<K, V>,
LegacyAsyncMap<K, V> {
```

Hazelcast - Mapa distribuido

El mapa distribuido en principio se puede usar como un mapa común de Java.

```java
public class DistributedMap {
    static void main() {
        Config config = new Config();
        // Rest of configuration
        HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
        Map<String, String> map = hz.getMap("my-distributed-map");
        map.put("key", "value");
        map.get("key");
        // Concurrent Map methods
        map.putIfAbsent("somekey", "somevalue");
        map.replace("key", "value", "newvalue");
    }
}
```

Hazelcast - Mapa distribuidos Procesamiento

Sobre el mapa se pueden realizar procesamiento atómico sobre entradas

```
public class EntryProcessorMain {
    static void main() {
        Config config = new Config();
        HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
        IMap<String, Integer> map = hz.getMap("map");
        map.put("key", 0);
        map.executeOnKey("key", new IncEntryProcessor());
        IO.println("new value: %d".formatted(map.get("key")));
    }
```

```
    private static class IncEntryProcessor
                        extends AbstractEntryProcessor<String, Integer> {
        @Override public Object process(Map.Entry<String, Integer> entry) {
            int oldValue = entry.getValue();
            int newValue = oldValue + 1;
            entry.setValue(newValue);
            return null;
        }
    }
```

```
new value: 1
```

Hazelcast - Map -> Query

Sobre el mapa realizar procesamientos “a la SQL” para consultas

```java
public class Query {
  static void main() {
      Config config = new Config();
      HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
      IMap<String, User> users = hz.getMap("users");
      generateRandomUsers(users); // Just generate some random users
      Predicate sqlQuery = new SqlPredicate("active AND age BETWEEN 18 AND 21");
      Predicate criteriaQuery = Predicates.and(
              Predicates.equal("active", true),
              Predicates.between("age", 18, 21)
      );
      Collection<User> result1 = users.values(sqlQuery);
      Collection<User> result2 = users.values(criteriaQuery);
  }
```

```java
public record User(String username, int age, boolean active)
      implements Serializable {}
```

Hazelcast - Queue

El funcionamiento de la clase Queue es también similar al trabajo local

```
public class DistributedQueue {
    static void main() throws InterruptedException {
        Config config = new Config();
        HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
        BlockingQueue<String> queue = hz.getQueue("my-distributed-queue");
        queue.offer("item");
        String item = queue.poll();
        //Timed blocking Operations
        queue.offer("anotheritem", 500, TimeUnit.MILLISECONDS);
        String anotherItem = queue.poll(5, TimeUnit.SECONDS);
        //Indefinitely blocking Operations
        queue.put("yetanotheritem");
        String yetanother = queue.take();
    }
}
```

Hazelcast - Topics

Los **tópicos** en Hazelcast permiten implementar una estrategia **pub/sub**

```
public class DistributedTopic implements MessageListener<String> {
    static void main() {
        Config config = new Config();
        HazelcastInstance hz = Hazelcast.newHazelcastInstance(config);
        ITopic<String> topic = hz.getTopic("my-distributed-topic");
        topic.addMessageListener(new DistributedTopic());
        topic.publish("Hello to distributed world");
    }
    @Override
    public void onMessage(Message<String> message) {
        IO.println("Got message " + message.getMessageObject());
    }
}
```

Hazelcast - Locks

Hazelcast permite tener *locks* distribuidos.

```
Lock testLock = hz.getLock("testLock");
testLock.lock();
try {
    // critical section code.
} finally {
    testLock.unlock();
}
```

```
Lock lock = hz.getLock(myLockedObject);
if (lock.tryLock(5000, TimeUnit.MILLISECONDS)) {
    try {
        // do some stuff here..
    } finally {
        lock.unlock();
    }
}
```

Hazelcast - Locks en Mapas

También se pueden generar *locks* a mapas

```
static void main() {
    HazelcastInstance hz = Hazelcast.newHazelcastInstance();
    IMap<String, Value> map = hz.getMap("map");
    String key = "1";
    map.put(key, new Value());
    for ( int k = 0; k < 1000; k++ ) {
        map.lock(key);
        try {
            // ...
        } finally {
            map.unlock(key);
        }
    }
}
```

Referencias y para profundizar

- https://hazelcast.com
- https://docs.hazelcast.org/docs/3.8.6/manual/html-single/index.html
- https://docs.hazelcast.org/docs/3.8.6/manual/html-single/index.html#configuring-declaratively
- https://docs.hazelcast.org/docs/3.8.6/manual/html-single/index.html#serialization-interface-types
- https://docs.hazelcast.com/imdg/3.12/network-partitioning/network-partitioning
- https://www.ecyrd.com/cassandracalculator/
- https://martin.kleppmann.com/2017/01/26/data-loss-in-large-clusters.html

CREDITS

Content of the slides:

- POD - ITBA

Images:

- POD - ITBA
- Or obtained from: commons.wikimedia.org

Slides theme credit:

Special thanks to all the people who made and released these awesome resources for free:

- Presentation template by SlidesCarnival
- Photographs by Unsplash